

UNIT I CONTROL FLOW, FUNCTIONS, LISTS, TUPLES

9

Python interpreter - data types, variables, expressions; operators, Conditionals: conditional (if), alternative (if-else), chained conditional (if-elif-else); Iteration: while, for, break, continue, pass; Fruitful functions: return values, parameters, local and global scope, function composition, recursion; Strings: String Operations; Lists as arrays, Lists: list operations, list slices, list methods, list loop, mutability, aliasing, cloning lists, list parameters advanced list processing - list comprehension; Tuples: tuple assignment, tuple as return value

1. Outline the modes Python interpreter works. (2 MARKS) (NOV 2017)
2. What is list in python? Give an example. (2 MARKS) (NOV 2017)
3. Difference between compiler and interpreter. (2 MARKS) (NOV 2019)

INTRODUCTION TO PYTHON:

Python is a general-purpose interpreted, interactive, object-oriented, and high-level programming language.

- It was created by Guido van Rossum during 1985- 1990.
- Python got its name from “Monty Python’s flying circus”. Python was released in the year 2000.
- **Python is interpreted:** Python is processed at runtime by the interpreter. You do not need to compile your program before executing it.
- **Python is Interactive:** You can actually sit at a Python prompt and interact with the interpreter directly to write your programs.
- **Python is Object-Oriented:** Python supports Object-Oriented style or technique of programming that encapsulates code within objects.
- **Python is a Beginner's Language:** Python is a great language for the beginner- level programmers and supports the development of a wide range of applications.

Python Features:

- **Easy-to-learn:** Python is clearly defined and easily readable. The structure of the program is very simple. It uses few keywords.
- **Easy-to-maintain:** Python's source code is fairly easy-to-maintain.
- **Portable:** Python can run on a wide variety of hardware platforms and has the same interface on all platforms.
- **Interpreted:** Python is processed at runtime by the interpreter. So, there is no need to compile a program before executing it. You can simply run the program.
- **Extensible:** Programmers can embed python within their C,C++,Java script ,ActiveX, etc.
- **Free and Open Source:** Anyone can freely distribute it, read the source code, and edit it.
- **High Level Language:** When writing programs, programmers concentrate on solutions of the current problem, no need to worry about the low level details.
- **Scalable:** Python provides a better structure and support for large programs than shell scripting.

Applications

- Bit Torrent file sharing
- Google search engine, Youtube
- Intel, Cisco, HP, IBM
- i-Robot
- NASA
- Facebook, Drop box

1. PYTHON INTERPRETER:

- **Interpreter:** To execute a program in a high-level language by translating it one line at a time.
- **Compiler:** To translate a program written in a high-level language into a low-level language all at once, in preparation for later execution.

Compiler	Interpreter
Compiler Takes Entire program as input	Interpreter Takes Single instruction as input
Intermediate Object Code is Generated	No Intermediate Object Code is Generated
Conditional Control Statements are Executes faster	Conditional Control Statements are Executes slower
Memory Requirement is More (Since Object Code is Generated)	Memory Requirement is Less
Program need not be compiled every time	Every time higher level program is converted into lower level program
Errors are displayed after entire program is checked	Errors are displayed for every instruction interpreted (if any)
Example : C Compiler	Example : PYTHON

MODES OF PYTHON INTERPRETER:

Python Interpreter is a program that reads and executes Python code. It uses 2 modes of Execution.

1. Interactive mode
2. Script mode

1. Interactive mode:

- Interactive Mode, as the name suggests, allows us to interact with OS.
- When we type Python statement, **interpreter displays the result(s) immediately.**
- **Advantages:**
 - Python, in interactive mode, is good enough to learn, experiment or explore.
 - Working in interactive mode is convenient for beginners and for testing small pieces of code.
- **Drawback:**
 - We cannot save the statements and have to retype all the statements once again to re-run them.

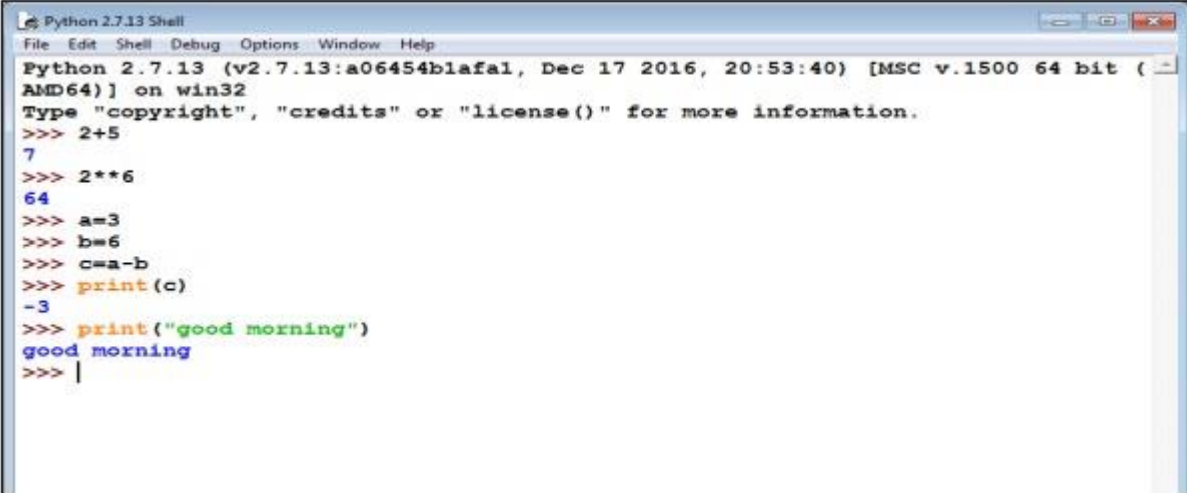
- In interactive mode, you type Python programs and the interpreter displays the result:

```
>>> 1 + 1
2
```

- The chevron, >>>, is the prompt the interpreter uses** to indicate that it is ready for you to enter code. If you type 1 + 1, the interpreter replies 2.

```
>>> print('Hello, World!')
Hello, World!
```

- This is an example of a print statement. It displays a result on the screen. In this case, the result is the words.



```
Python 2.7.13 Shell
File Edit Shell Debug Options Window Help
Python 2.7.13 (v2.7.13:a06454blafal, Dec 17 2016, 20:53:40) [MSC v.1500 64 bit (AMD64)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> 2+5
7
>>> 2**6
64
>>> a=3
>>> b=6
>>> c=a-b
>>> print(c)
-3
>>> print("good morning")
good morning
>>> |
```

2. Script mode:

- In script mode, we type python program in a file and then use interpreter to execute the content of the file.
- Scripts can be saved to disk for future use. **Python scripts have the extension .py**, meaning that the filename ends with .py
- Save the code with **filename.py** and run the interpreter in script mode to execute the script.

Example1:

```
print(1)
x = 2
print(x)
```

Output:

```
>>>1
2
```

Interactive mode	Script mode
A way of using the Python interpreter by typing commands and expressions at the prompt.	A way of using the Python interpreter to read and execute statements in a script.
Cant save and edit the code	Can save and edit the code
If we want to experiment with the code, we can use interactive mode.	If we are very clear about the code, we can use script mode.
we cannot save the statements for further use and we have to retype all the statements to re-run them.	we can save the statements for further use and we no need to retype all the statements to re-run them.
We can see the results immediately.	We cant see the code immediately.

Integrated Development Learning Environment (IDLE):

- Is a **graphical user interface** which is completely written in Python.
- It is bundled with the default implementation of the python language and also comes with optional part of the Python packaging.
- **Features of IDLE:**
 - Multi-window text editor with syntax highlighting.
 - Auto completion with smart indentation.
 - **Python shell** to display output with syntax highlighting.

2. VALUES AND DATA TYPES

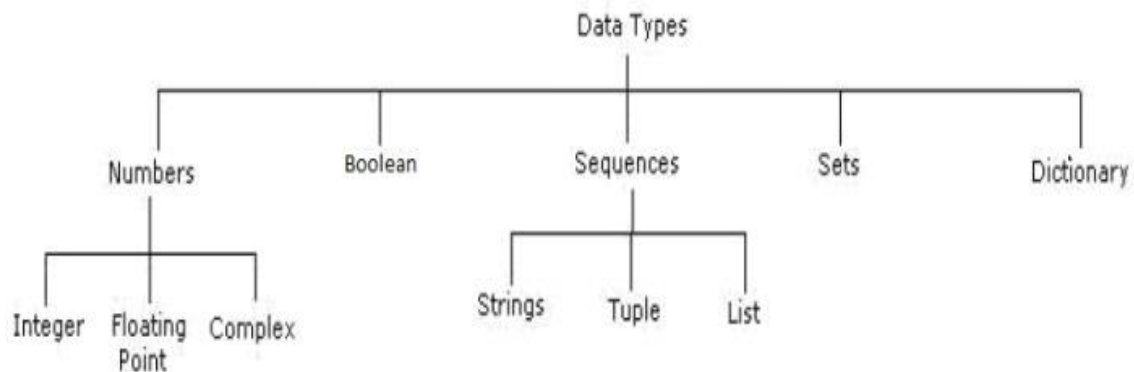
1. Explain the various data types in Python. (8 MARKS) (NOV 2017)
2. What are keywords? Give examples. (2 MARKS) (JAN 2019)
3. Mention the list of keywords available in python. Compare it with variable name. (8 MARKS) (APR2019)
4. What are statements? How they are constructed from variables and expressions in python? (8 MARKS) (APR2019)
5. Outline the data types supported by python with an example. (8 MARKS) (JAN 2022)
6. Are comments executable statements in a python program? How comments are included in python program? (2 MARKS) (JAN 2022)

Value:

- **Value** can be any letter ,number or string.
- **Eg,** Values are 2, 42.0, and 'Hello, World!'. (These values belong to different datatypes.)

Data type:

- Every value in Python has a data type.
- It is a set of values, and the allowable operations on those values.
- **Python has four standard data types:**



Numbers:

- Number data type stores **Numerical Values**.
- This data type is immutable [i.e. values/items cannot be changed].
- Python supports integers, floating point numbers and complex numbers. They are defined as,

Integers	Long	Float	Complex
- They are often called just integers or <code>int</code>. - They are positive or negative whole numbers with no decimal point.	-They are long integers. -They can also be represented in octal and hexadecimal representation.	-They are written with a decimal point dividing the integer and the fractional parts.	-They are of the form $a + bj$, where a and b are floats and j represents the square root of -1 (which is an imaginary number). -The real part of the number is a, and the imaginary part is b.
Eg, 56	Eg, 5692431L	Eg, 56.778	Eg, square root of -1 is a complex number

Sequence:

- A sequence is an **ordered collection of items**, indexed by positive integers.
- It is a combination of **mutable** (value can be changed) and **immutable** (values cannot be changed) data types.
- There are three types of sequence data type available in Python, they are
 - Strings**
 - Lists**
 - Tuples**

1. Strings

- A String in Python consists of a series or sequence of characters - letters, numbers, and special characters.
- Strings are marked by quotes:
 - single quotes (' ') Eg, "This a string in single quotes"
 - double quotes (" ") Eg, ""This a string in double quotes""
 - triple quotes (" " " " " ") Eg, This is a paragraph. It is made up of multiple lines and sentences. " " " " " "
- individual character in a string is accessed using a subscript (index).
- Characters can be accessed using indexing and slicing operations
- Strings are immutable i.e. the contents of the string cannot be changed after it is created.
- **Indexing:**

String A	H	E	L	L	O
Positive Index	0	1	2	3	4
Negative Index	-5	-4	-3	-2	-1

- Positive indexing helps in accessing the string from the beginning
- Negative subscript helps in accessing the string from the end.
- Subscript 0 or -ve n (where n is length of the string) displays the first element.

Example: A[0] or A[-5] will display "H"

Subscript 1 or -ve (n-1) displays the second element.

Example: A[1] or A[-4] will display "E"

Operations on string:

- Indexing
- Slicing
- Concatenation
- Repetitions
- Member ship

Creating a string	>>> s="good morning"	Creating the list with elements of different data types.
Indexing	>>> print(s[2]) 0 >>> print(s[6]) 0	❖ Accessing the item in the position 0 ❖ Accessing the item in the position 2
Slicing(ending position -1)	>>> print(s[2:]) od morning	- Displaying items from 2 nd till last.
<u>Slice operator is used to extract part of a data type</u>	>>> print(s[:4]) Good	- Displaying items from 1 st position till 3 rd .
Concatenation	>>> print(s+"friends") good morningfriends	-Adding and printing the characters of two strings.
Repetition	>>> print(s*2) good morninggood morning	Creates new strings, concatenating multiple copies of the same string
in, not in (membership operator)	>>> s="good morning" >>> "m" in s True >>> "a" not in s True	Using membership operators to check a particular character is in string or not. Returns true if present.

2. Lists

- List is an ordered sequence of items. Values in the list are called elements / items.
- It can be written as a list of comma-separated items (values) between **square brackets []**.
- Items in the lists can be of different data types.
- **Operations on list:**
 - Indexing
 - Slicing
 - Concatenation
 - Repetitions
 - Updation, Insertion, Deletion

Creating a list	<pre>>>>list1=["python", 7.79, 101, "hello"] >>>list2=["god",6.78,9]</pre>	Creating the list with elements of different data types.
Indexing	<pre>>>>print(list1[0]) python >>> list1[2] 101</pre>	❖ Accessing the item in the position 0 ❖ Accessing the item in the position 2
Slicing(ending position -1) <u>Slice operator is used to extract part of a string, or some part of a list</u> <u>Python</u>	<pre>>>> print(list1[1:3]) [7.79, 101] >>>print(list1[1:]) [7.79, 101, 'hello']</pre>	- Displaying items from 1st till 2nd. - Displaying items from 1 st position till last.
Concatenation	<pre>>>>print(list1+list2) ['python', 7.79, 101, 'hello', 'god', 6.78, 9]</pre>	-Adding and printing the items of two lists.
Repetition	<pre>>>> list2*3 ['god', 6.78, 9, 'god', 6.78, 9, 'god', 6.78, 9]</pre>	Creates new strings, concatenating multiple copies of the same string
Updating the list	<pre>>>> list1[2]=45 >>>print(list1) ['python', 7.79, 45, 'hello']</pre>	Updating the list using index value
Inserting an element	<pre>>>> list1.insert(2,"program") >>> print(list1) ['python', 7.79, 'program', 45, 'hello']</pre>	Inserting an element in 2 nd position
Removing an element	<pre>>>> list1.remove(45) >>> print(list1) ['python', 7.79, 'program', 'hello']</pre>	Removing an element by giving the element directly

3. Tuple:

- A tuple is same as list, except that the set of elements is **enclosed in parentheses** instead of square brackets.
- **A tuple is an immutable list.** i.e. once a tuple has been created, you can't add elements to a tuple or remove elements from the tuple.
- Benefit of Tuple:
- Tuples are faster than lists.
- If the user wants to protect the data from accidental changes, tuple can be used.
- Tuples can be used as keys in dictionaries, while lists can't.
- **Basic Operations:**

Creating a tuple	<pre>>>>t=("python", 7.79, 101, "hello")</pre>	Creating the tuple with elements of different data types.
Indexing	<pre>>>>print(t[0]) python >>> t[2] 101</pre>	❖ Accessing the item in the position 0 ❖ Accessing the item in the position 2
Slicing(ending position -1)	<pre>>>>print(t[1:3]) (7.79, 101)</pre>	❖ Displaying items from 1st till 2nd.
Concatenation	<pre>>>> t+("ram", 67) ('python', 7.79, 101, 'hello', 'ram', 67)</pre>	❖ Adding tuple elements at the end of another tuple elements
Repetition	<pre>>>>print(t*2) ('python', 7.79, 101, 'hello', 'python', 7.79, 101, 'hello')</pre>	❖ Creates new strings, concatenating multiple copies of the same string

- **Altering the tuple data type leads to error.** Following error occurs when user tries to do.
- ```
>>> t[0]="a"
```
- Trace back (most recent call last):
- File "<stdin>", line 1, in <module>
- Type Error: 'tuple' object does not support item assignment

### Mapping

- This data type is unordered and mutable.
- Dictionaries fall under Mappings.

### Dictionaries:

- Lists are ordered sets of objects, whereas **dictionaries are unordered sets.**
- Dictionary is created by using **curly brackets**. i.e. {}
- Dictionaries **are accessed via keys** and not via their position.
- A dictionary is an associative array (also known as hashes). Any key of the dictionary is associated (or mapped) to a value.
- The values of a dictionary can be any Python data type. So dictionaries are
- **unordered key-value-pairs**(The association of a key and a value is called a key-value pair )
- Dictionaries don't support the sequence operation of the sequence data types like strings, tuples and lists.

|                              |                                                                                                                                                    |                                                                |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------|
| Creating a dictionary        | <pre>&gt;&gt;&gt; food = {"ham":"yes", "egg" : "yes", "rate":450 }<br/>&gt;&gt;&gt;print(food)<br/>{'rate': 450, 'egg': 'yes', 'ham': 'yes'}</pre> | Creating the dictionary with elements of different data types. |
| Indexing                     | <pre>&gt;&gt;&gt;&gt; print(food["rate"])<br/>450</pre>                                                                                            | Accessing the item with keys.                                  |
| Slicing( ending position -1) | <pre>&gt;&gt;&gt;print(t[1:3])<br/>(7.79, 101)</pre>                                                                                               | Displaying items from 1st till 2nd.                            |

- If you try to access a key which doesn't exist, you will get an error message:

- `>>> words = {"house": "Haus", "cat": "Katze"}`
- `>>> words["car"]`
- Traceback (most recent call last):
- File "<stdin>", line 1, in <module>
- `KeyError: 'car'`

| Data type | Compile time                  | Run time                                     |
|-----------|-------------------------------|----------------------------------------------|
| int       | <code>a=10</code>             | <code>a=int(input("enter a"))</code>         |
| float     | <code>a=10.5</code>           | <code>a=float(input("enter a"))</code>       |
| string    | <code>a= "RIT CHENNAI"</code> | <code>a=input("enter a string")</code>       |
| list      | <code>a=[20,30,40,50]</code>  | <code>a=list(input("enter a list"))</code>   |
| tuple     | <code>a=(20,30,40,50)</code>  | <code>a=tuple(input("enter a tuple"))</code> |

## 1. VARIABLES:

- A variable allows us to store a value by assigning it to a name, which can be used later.
- Named memory locations to store values.
- Programmers generally choose names for their variables that are meaningful.
- It can be of any length. No space is allowed.
- We don't need to declare a variable before using it. In Python, we simply assign a value to a variable and it will exist.

### Assigning value to variable:

- Value should be given on the right side of assignment operator (=) and variable on left side.  
`>>> counter = 45`  
`print(counter)`
- Assigning a single value to several variables simultaneously:  
`>>> a=b=c=100`
- Assigning multiple values to multiple variables:  
`>>> a,b,c=2,4,"ram"`

### KEYWORDS:

- Keywords are the reserved words in Python.
- We cannot use a keyword as variable name, function name or any other identifier.
- They are used to define the syntax and structure of the Python language.
- Keywords are case sensitive.

|               |                 |                |                 |               |
|---------------|-----------------|----------------|-----------------|---------------|
| <i>False</i>  | <i>class</i>    | <i>finally</i> | <i>is</i>       | <i>return</i> |
| <i>None</i>   | <i>continue</i> | <i>for</i>     | <i>lambda</i>   | <i>try</i>    |
| <i>True</i>   | <i>def</i>      | <i>from</i>    | <i>nonlocal</i> | <i>while</i>  |
| <i>and</i>    | <i>del</i>      | <i>global</i>  | <i>not</i>      | <i>with</i>   |
| <i>as</i>     | <i>elif</i>     | <i>if</i>      | <i>or</i>       | <i>yield</i>  |
| <i>assert</i> | <i>else</i>     | <i>import</i>  | <i>pass</i>     |               |
| <i>break</i>  | <i>except</i>   | <i>in</i>      | <i>raise</i>    |               |

## 2. IDENTIFIERS:

- **Identifier is the name given to entities like class, functions, variables etc. in Python.**
- Identifiers can be a combination of letters in lowercase (a to z) or uppercase (A to Z) or digits (0 to 9) or an underscore (\_).
- all are valid example.
- An identifier cannot start with a digit.
- Keywords cannot be used as identifiers.
- Cannot use special symbols like !, @, #, \$, % etc. in our identifier.
- Identifier can be of any length.
- **Example:**
  - Names like myClass, var\_1, and **this\_is\_a\_long\_variable**

| Valid declarations | Invalid declarations |
|--------------------|----------------------|
| Num                | Number 1             |
| Num                | num 1                |
| Num1               | addition of program  |
| _NUM               | 1Num                 |
| NUM_temp2          | Num.no               |
| IF                 | if                   |
| Else               | else                 |

### 3. INPUT AND OUTPUT

- **INPUT:** Input is data entered by user (end user) in the program.
  - In python, **input () function** is available for input.
  - **Syntax for input() is:**  
variable = input ("data")
  - **Example:**  

```
>>> x=input("enter the name:") enter the name: george
>>>y=int(input("enter the number"))
enter the number 3
#python accepts string as default data type. conversion is required for type.
```
- **OUTPUT:** Output can be displayed to the user using Print statement .
  - **Syntax:**  
print (expression/constant/variable)
  - **Example:**  
print ("Hello") Hello

#### **COMMENTS:**

- A **hash sign (#)** is the beginning of a comment.
- Anything written after # in a line is ignored by interpreter.
- **Eg:**percentage = (minute \* 100) / 60      **# calculating percentage of an hour**
- Python **does not have multiple-line commenting feature.**
- You have to comment each line individually as follows :
- **Example:**

```
This is a comment.
This is a comment, too.
I said that already.
```

#### **LINES AND INDENTATION:**

- Most of the programming languages like C, C++, Java use braces { } to define a block of code. But, python uses indentation.
- Blocks of code are denoted by line indentation.
- It is a space given to the block of codes for class and function definitions or flow control.

- **Example:**

```
a=3
b=1
if a>b:
 print("a is greater")
else:
 print("b is greater")
```

#### 4. **OPERATORS:**

1. Outline with an example the assignment operators supported in Python.(8 MARKS) (NOV 2017)
2. State about logical operators available in python with an example. (2 MARKS) (APR 2019)
3. Describe the concept of precedence and associativity of operators with example. (8 MARKS) (APR2019)
4. Name the types of operators supported by python and Outline any two with an example. (8 MARKS) (JAN 2022)
5. In python, how the values stored in list are accessed? Should the elements of a list be of the same data type? (2 MARKS) (JAN 2022)

- Operators are the constructs which can manipulate the value of operands.
- Consider the **expression  $4 + 5 = 9$** . Here, **4 and 5 are called operands** and **+ is called operator**

##### **Types of Operators:**

- Python language supports the following types of operators
  - Arithmetic Operators
  - Comparison (Relational) Operators
  - Assignment Operators
  - Logical Operators
  - Bitwise Operators
  - Membership Operators
  - Identity Operators

##### **Arithmetic operators:**

- They are used to perform **mathematical operations** like addition, subtraction, multiplication etc. **Assume, a=10 and b=5**

| Operator         | Description                                                                                                                        | Example                       |
|------------------|------------------------------------------------------------------------------------------------------------------------------------|-------------------------------|
| + Addition       | Adds values on either side of the operator.                                                                                        | $a + b = 30$                  |
| - Subtraction    | Subtracts right hand operand from left hand operand.                                                                               | $a - b = -10$                 |
| * Multiplication | Multiplies values on either side of the operator                                                                                   | $a * b = 200$                 |
| / Division       | Divides left hand operand by right hand operand                                                                                    | $b / a = 2$                   |
| % Modulus        | Divides left hand operand by right hand operand and returns remainder                                                              | $b \% a = 0$                  |
| ** Exponent      | Performs exponential (power) calculation on operators                                                                              | $a ** b = 10$ to the power 20 |
| //               | Floor Division - The division of operands where the result is the quotient in which the digits after the decimal point are removed | $5 // 2 = 2$                  |

#### Example program

```
a=10
b=5
print("a+b=",a+b)
print("a-b=",a-b)
print("a*b=",a*b)
print("a/b=",a/b)
print("a%b=",a%b)
print("a//b=",a//b)
print("a**b=",a**b)
```

#### Output:

```
a+b= 15
a-b= 5
a*b= 50
a/b= 2.0
a%b= 0
a//b= 2
a**b= 100000
```

#### Comparison (Relational) Operators:

- Comparison operators are used to compare values.
- It either returns True or False according to the condition. **Assume, a=10 and b=5**

| Operator | Description                                                                                                       | Example               |
|----------|-------------------------------------------------------------------------------------------------------------------|-----------------------|
| ==       | If the values of two operands are equal, then the condition becomes true.                                         | (a == b) is not true. |
| !=       | If values of two operands are not equal, then condition becomes true.                                             | (a != b) is true      |
| >        | If the value of left operand is greater than the value of right operand, then condition becomes true.             | (a > b) is not true.  |
| <        | If the value of left operand is less than the value of right operand, then condition becomes true.                | (a < b) is true.      |
| >=       | If the value of left operand is greater than or equal to the value of right operand, then condition becomes true. | (a >= b) is not true. |
| <=       | If the value of left operand is less than or equal to the value of right operand, then condition becomes true.    | (a <= b) is true.     |

### Example

```
a=10
b=5
print("a>b=>",a>b)
print("a>b=>",a<b)
print("a==b=>",a==b)
print("a!=b=>",a!=b)
print("a>=b=>",a<=b)
print("a>=b=>",a>=b)
```

### Output:

```
a>b=> True
a>b=> False
a==b=> False
a!=b=> True
a>=b=> False
a>=b=> True
```

### Assignment Operators:

- Assignment operators are used in Python to assign values to variables.

| Operator              | Description                                                                                | Example                                                                                    |
|-----------------------|--------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| =                     | Assigns values from right side operands to left side operand                               | $c = a + b$<br>assigns<br>value of $a + b$<br>into $c$                                     |
| += Add AND            | It adds right operand to the left operand and assign the result to left operand            | $c += a$ is<br>equivalent<br>to $c = c + a$                                                |
| -= Subtract<br>AND    | It subtracts right operand from the left operand and assign the result to left operand     | $c -= a$ is<br>equivalent<br>to $c = c - a$                                                |
| *= Multiply<br>AND    | It multiplies right operand with the left operand and assign the result to left operand    | $c *= a$ is<br>equivalent<br>to $c = c * a$                                                |
| /= Divide<br>AND      | It divides left operand with the right operand and assign the result to left operand       | $c /= a$ is<br>equivalent<br>to $c = c / a$<br>$c /= a$ is<br>equivalent<br>to $c = c / a$ |
| %= Modulus<br>AND     | It takes modulus using two operands and assign the result to left operand                  | $c \% = a$ is<br>equivalent<br>to $c = c \% a$                                             |
| **= Exponent<br>AND   | Performs exponential (power) calculation on operators and assign value to the left operand | $c ** = a$ is<br>equivalent<br>to $c = c ** a$                                             |
| //= Floor<br>Division | It performs floor division on operators and assign value to the left operand               | $c //= a$ is<br>equivalent<br>to $c = c // a$                                              |

### Example

```

a = 21
b = 10
c = 0
c = a + b
print("Line 1 - Value of c is ", c)
c += a
print("Line 2 - Value of c is ", c)
c *= a
print("Line 3 - Value of c is ", c)

```

```
c /= a
print("Line 4 - Value of c is ", c)
c = 2 * c % a
print("Line 5 - Value of c is ", c) c **= a
print("Line 6 - Value of c is ", c) c // = a
print("Line 7 - Value of c is ", c)
```

### Output

```
Line 1 - Value of c is 31
Line 2 - Value of c is 52
Line 3 - Value of c is 1092
Line 4 - Value of c is 52.0
Line 5 - Value of c is 2
Line 6 - Value of c is 2097152
Line 7 - Value of c is 99864
```

### Logical Operators:

- Logical operators are the and, or, not operators.

| Operator | Meaning                                            | Example |
|----------|----------------------------------------------------|---------|
| and      | True if both the operands are true                 | x and y |
| or       | True if either of the operands is true             | x or y  |
| not      | True if operand is false (complements the operand) | not x   |

### Example

```
a = True
b = False
print('a and b is',a and b)
print('a or b is',a or b)
print('not a is',not a)
```

### Output

```
x and y is False
x or y is True
not x is False
```

### Bitwise Operators:

- A **bitwise operation** operates on one or more **bit** patterns at the level of individual Bits

### Example:

```
Let x = 10 (0000 1010 in binary) and
y = 4 (0000 0100 in binary)
```

| Operator | Meaning             | Example                   |
|----------|---------------------|---------------------------|
| &        | Bitwise AND         | x & y = 0 ( 0000 0000 )   |
|          | Bitwise OR          | x   y = 14 ( 0000 1110 )  |
| ~        | Bitwise NOT         | ~x = -11 ( 1111 0101 )    |
| ^        | Bitwise XOR         | x ^ y = 14 ( 0000 1110 )  |
| >>       | Bitwise right shift | x >> 2 = 2 ( 0000 0010 )  |
| <<       | Bitwise left shift  | x << 2 = 40 ( 0010 1000 ) |

### Example

```

a = 60 # 60 = 0011 1100
b = 13 # 13 = 0000 1101
c = 0
c = a & b; # 12 = 0000 1100
print "Line 1 - Value of c is ", c
c = a | b; # 61 = 0011 1101
print "Line 2 - Value of c is ", c
c = a ^ b; # 49 = 0011 0001
print "Line 3 - Value of c is ", c
c = ~a; # -61 = 1100 0011
print "Line 4 - Value of c is ", c
c = a << 2; # 240 = 1111 0000
print "Line 5 - Value of c is ", c
c = a >> 2; # 15 = 0000 1111
print "Line 6 - Value of c is ", c

```

### Output

```

Line 1 - Value of c is 12
Line 2 - Value of c is 61
Line 3 - Value of c is 49
Line 4 - Value of c is -61
Line 5 - Value of c is 240
Line 6 - Value of c is 15

```

### Membership Operators:

- Evaluates to find a value or a variable is in the specified sequence of string, list, tuple, dictionary or not.
- v Let, **x=[5,3,6,4,1]**. To check particular item in list or not, **in** and **not in** operators are used.

| Operator | Meaning                                             | Example    |
|----------|-----------------------------------------------------|------------|
| in       | True if value/variable is found in the sequence     | 5 in x     |
| not in   | True if value/variable is not found in the sequence | 5 not in x |

Example:

```
x=[5,3,6,4,1]
>>> 5 in x
True
>>> 5 not in x
False
```

### Identity Operators

- They are used to check if two values (or variables) are located on the same part of the memory.

| Operator | Meaning                                                                  | Example       |
|----------|--------------------------------------------------------------------------|---------------|
| is       | True if the operands are identical (refer to the same object)            | x is True     |
| is not   | True if the operands are not identical (do not refer to the same object) | x is not True |

Example

```
x = 5
y = 5
x2 = 'Hello'
y2 = 'Hello'
print(x1 is not y1)
print(x2 is y2)
```

Output

```
False
True
```

## 5. CONDITIONALS

- Conditional if
- Alternative if... else
- Chained if...elif...else
- Nested if....else

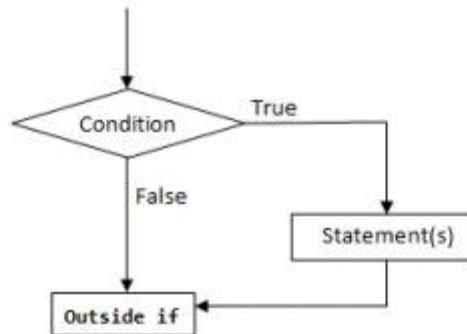
**Conditional (if):**

- conditional (if) is used to test a condition, if the condition is true the statements inside if will be executed.

**syntax:**

**if(condition 1):**  
**Statement 1**

**Flowchart:**



**Example:**

1. Program to provide flat rs 500, if the purchase amount is greater than 2000.
2. Program to provide bonus mark if the category is sports.

**Program to provide flat rs 500, if the purchase amount is greater than 2000.**

```

purchase=eval(input("enter your purchase amount"))
if(purchase>=2000):
 purchase=purchase-500
print("amount to pay",purchase)

```

**output**

```

enter your purchase
amount
2500
amount to pay
2000

```

**Program to provide bonus mark if the category is sports**

```

m=eval(input("enter ur mark out of 100"))
c=input("enter ur category G/S")
if(c=="S"):
 m=m+5
print("mark is",m)

```

**output**

```

enter ur mark out of 100
85
enter ur category G/S
S
mark is 90

```

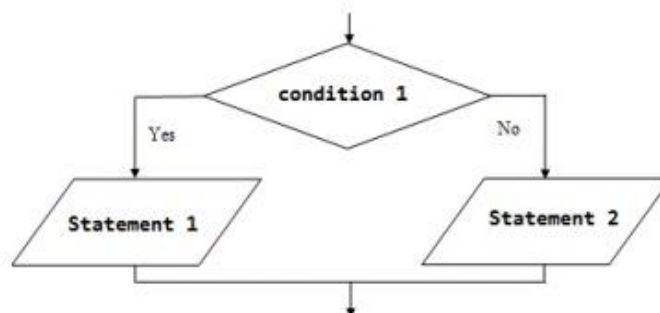
**alternative (if-else)**

In the alternative the condition must be true or false. In this **else** statement can be combined with **if** statement. The **else** statement contains the block of code that executes when the condition is false. If the condition is true statements inside the if get executed otherwise else part gets executed. The alternatives are called branches, because they are branches in the flow of execution.

**syntax:**

```
if(condition 1):
 Statement 1
else:
 Statement 2
```

**Flowchart:**



**Examples:**

1. odd or even number
2. positive or negative number
3. leap year or not
4. greatest of two numbers
5. eligibility for voting

**Odd or even number**

```
n=eval(input("enter a number"))
if(n%2==0):
 print("even number")
else:
 print("odd number")
```

**Output**

```
enter a number4
even number
```

**positive or negative number**

```
n=eval(input("enter a number"))
if(n>=0):
 print("positive number")
else:
 print("negative number")
```

**Output**

enter a number8  
positive number

### leap year or not

```
y=eval(input("enter a yaer"))
if(y%4==0):
 print("leap year")
else:
 print("not leap year")
```

### Output

enter a yaer2000  
leap year

### greatest of two numbers

```
a=eval(input("enter a value:"))
b=eval(input("enter b value:"))
if(a>b):
 print("greatest:",a)
else:
 print("greatest:",b)
```

### Output

enter a value:4  
enter b value:7  
greatest: 7

### eligibility for voting

```
age=eval(input("enter ur age:"))
if(age>=18):
 print("you are eligible for vote")
else:
 print("you are eligible for vote")
```

### Output

enter ur age:78  
you are eligible for vote

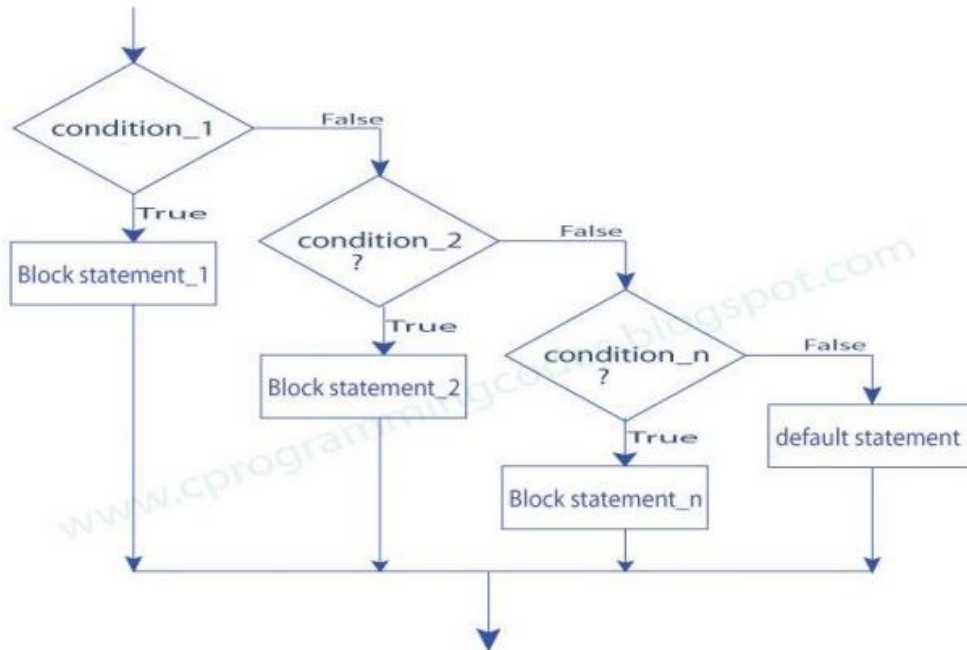
### Chained conditionals(if-elif-else)

- The elif is short for else if.
- This is used to check more than one condition.
- If the condition1 is False, it checks the condition2 of the elif block. If all the conditions are False, then the else part is executed.
- Among the several if...elif...else part, only one part is executed according to the condition.
- The if block can have only one else block. But it can have multiple elif blocks.
- The way to express a computation like that is a chained conditional.

### syntax:

```
if(condition 1):
 statement 1
elif(condition 2):
 statement 2
elif(condition 3):
 statement 3
else:
 default statement
```

**Flowchart:**



**Example:**

1. student mark system
2. traffic light system
3. compare two numbers
4. roots of quadratic equation

**student mark system**

```
mark=eval(input("enter ur mark:"))
if(mark>=90):
 print("grade:S")
elif(mark>=80):
 print("grade:A")
elif(mark>=70):
 print("grade:B")
elif(mark>=50):
 print("grade:C")
else:
 print("fail")
```

**Output**

enter ur mark:78

grade:B

### traffic light system

```
colour=input("enter colour of light:")
if(colour=="green"):
 print("GO")
elif(colour=="yellow"):
 print("GET READY")
else:
 print("STOP")
```

### Output

```
enter colour of light:green
GO
```

### compare two numbers

```
x=eval(input("enter x value:"))
y=eval(input("enter y value:"))
if(x == y):
 print("x and y are equal")
elif(x < y):
 print("x is less than y")
else:
 print("x is greater than y")
```

### Output

```
enter x value:5
enter y value:7
x is less than y
```

### Roots of quadratic equation

```
a=eval(input("enter a value:"))
b=eval(input("enter b value:"))
c=eval(input("enter c value:"))
d=(b*b-4*a*c)
if(d==0):
 print("same and real roots")
elif(d>0):
 print("diffrent real roots")
else:
 print("imaginagry roots")
```

### output

```
enter a value:1
enter b value:0
enter c value:0
same and real roots
```

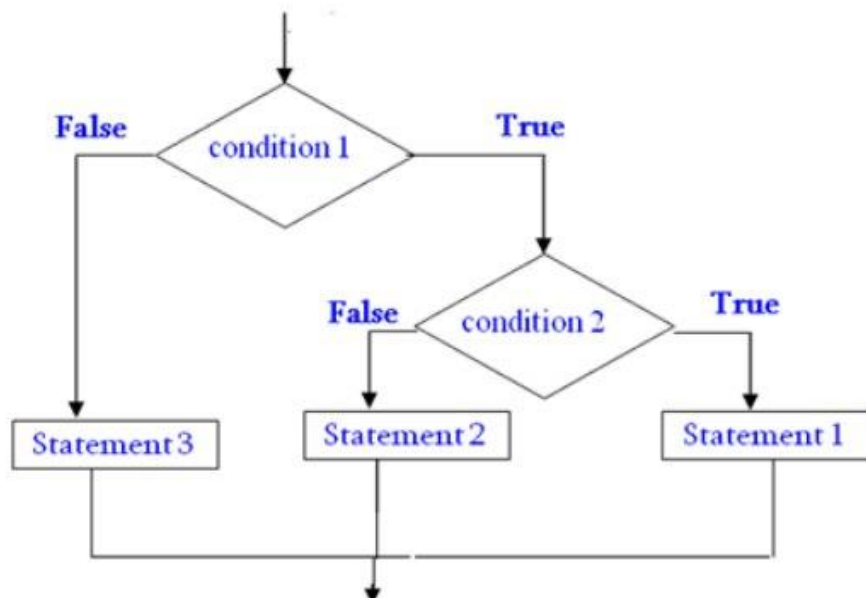
### Nested conditionals

- One conditional can also be nested within another. Any number of condition can be nested inside one another. In this, if the condition is true it checks another if condition1. If both the conditions are true statement1 get executed otherwise statement2 get execute. if the condition is false statement3 gets executed

**Syntax:**

```
if (condition):
 if(condition 1):
 statement 1
 else:
 statement 2
else:
 statement 3
```

**Flowchart:**



**Exampe:**

1. greatest of three numbers
2. positive negative or zero

**greatest of three numbers**

```
a=eval(input("enter the value of a"))
b=eval(input("enter the value of b"))
c=eval(input("enter the value of c"))
if(a>b):
 if(a>c):
 print("the greatest no is",a)
 else:
 print("the greatest no is",c)
else:
 if(b>c):
```

```
print("the greatest no is",b)
else:
 print("the greatest no is",c)
```

**output**

enter the value of a 9  
enter the value of a 1  
enter the value of a 8  
the greatest no is 9

**positive negative or zero**

```
n=eval(input("enter the value of n:"))
if(n==0):
 print("the number is zero")
else:
 if(n>0):
 print("the number is positive")
 else:
 print("the number is negative")
```

**output**

enter the value of n:-9  
the number is negative

**6. ITERATION/CONTROL STATEMENTS:**

1. Write a Python program to accept two numbers, multiply them and print the result. (2 MARKS) (JAN 2018)
2. Explain while loop with example. (or) Explain flow of execution of while loop with Example. (2 MARKS) (JAN 2019)
3. Discuss conditional alternative and chained conditional in detail. (16MARKS) (NOV 2019)
4. Outline the conditional breaking statements in python with an example. (16 MARKS) (JAN 2022)
5. Outline the while loop, break statement and continue statement in python with example. (12 MARKS) (JAN 2022)
6. Name the two types of iterative function supported by python. (12 MARKS) (JAN 2022)

- state
- while
- for
- break
- continue
- pass

**State:**

Transition from one process to another process under specified condition with in a time is called state.

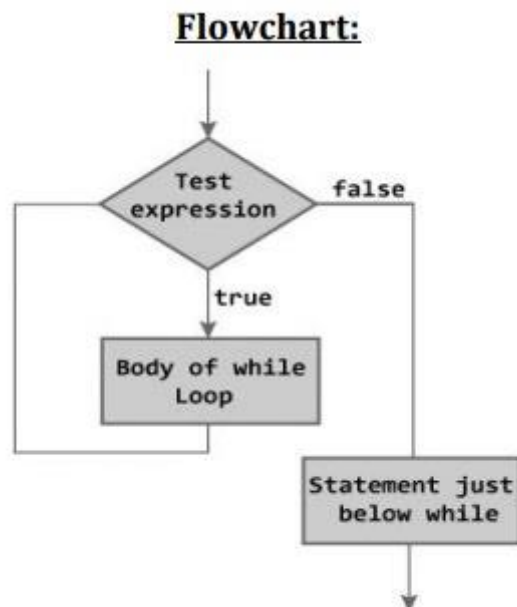
**While loop:**

- While loop statement in Python is used to repeatedly executes set of statement as long as a given condition is true.
- In while loop, test expression is checked first. The body of the loop is entered only if the test\_expression is True. After one iteration, the test expression is checked again. This process continues until the test\_expression evaluates to False.
- In Python, the body of the while loop is determined through indentation.
- The statements inside the while starts with indentation and the first unindented line marks the end.

**Syntax:**

```
initial value
while(condition):
 body of while loop
increment
```

**Flowchart**



**Examples:**

1. program to find sum of n numbers:
2. program to find factorial of a number
3. program to find sum of digits of a number:
4. Program to Reverse the given number:
5. Program to find number is Armstrong number or not
6. Program to check the number is palindrome or not

**Sum of n numbers:**

```
n=eval(input("enter n"))
i=1
sum=0
while(i<=n):
 sum=sum+i
 i=i+1
print(sum)
```

**output**

```
enter n
10
55
```

**Factorial of a numbers:**

```
n=eval(input("enter n"))
i=1
fact=1
while(i<=n):
 fact=fact*i
 i=i+1
print(fact)
```

**output**

```
enter n
5
120
```

**Sum of digits of a number:**

```
n=eval(input("enter a number"))
sum=0
while(n>0):
 a=n%10
 sum=sum+a
 n=n//10
print(sum)
```

**output**

```
enter a number
123
6
```

**Reverse the given number:**

```
n=eval(input("enter a number"))
sum=0
while(n>0):
 a=n%10
 sum=sum*10+a
 n=n//10
print(sum)
```

**output**

```
enter a number
123
321
```

**Armstrong number or not**

```
n=eval(input("enter a number"))
```

```
org=n
sum=0
while(n>0):
 a=n%10
 sum=sum+a*a*a
 n=n//10
if(sum==org):
 print("The given number is Armstrong number")
else:
 print("The given number is not Armstrong number")
```

#### **output**

```
enter a number153
The given number is Armstrong number
```

#### **Palindrome or not**

```
n=eval(input("enter a number"))
org=n
sum=0
while(n>0):
 a=n%10
 sum=sum*10+a
 n=n//10
if(sum==org):
 print("The given no is palindrome")
else:
 print("The given no is not palindrome")
```

#### **output**

```
enter a number121
The given no is palindrome
```

#### **For loop:**

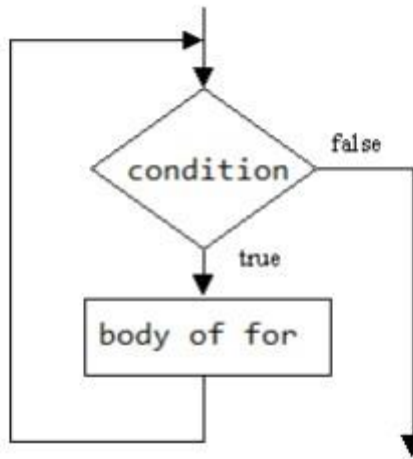
##### **for in range:**

- We can generate a sequence of numbers using range() function. range(10) will generate numbers from 0 to 9 (10 numbers).
- In range function have to define the start, stop and step size as range(start,stop,step size). step size defaults to 1 if not provided.

##### **yntax**

```
for i in range(start,stop,steps):
 body of for loop
```

#### **Flowchart:**



### For in sequence

- The for loop in Python is used to iterate over a sequence (list, tuple, string). Iterating over a sequence is called traversal. Loop continues until we reach the last element in the sequence.
- The body of for loop is separated from the rest of the code using indentation.

```

for i in sequence:
 print(i)

```

### Sequence can be a list, strings or tuples

| s.no | sequences          | example                           | output                |
|------|--------------------|-----------------------------------|-----------------------|
| 1.   | For loop in string | for i in "Ramu":<br>print(i)      | R<br>A<br>M<br>U      |
| 2.   | For loop in list   | for i in [2,3,5,6,9]:<br>print(i) | 2<br>3<br>5<br>6<br>9 |
| 3.   | For loop in tuple  | for i in (2,3,1):<br>print(i)     | 2<br>3<br>1           |

### **Examples:**

1. print nos divisible by 5 not by 10:
2. Program to print fibonacci series.
3. Program to find factors of a given number
4. check the given number is perfect number or not

### **print nos divisible by 5 not by 10**

```

n=eval(input("enter a"))
for i in range(1,n,1):
 if(i%5==0 and i%10!=0):

```

```
print(i)
```

**output**

```
enter a:30
5
15
25
```

**Fibonacci series**

```
a=0
b=1
n=eval(input("Enter the number of terms: "))
print("Fibonacci Series: ")
print(a,b)
for i in range(1,n,1):
 c=a+b
 print(c)
 a=b
 b=c
```

**output**

```
Enter the number of terms: 6
Fibonacci Series:
0 1
1
2
3
5
8
```

**find factors of a number**

```
n=eval(input("enter a number:"))
for i in range(1,n+1,1):
 if(n%i==0):
 print(i)
```

**Output**

```
enter a number:10
1
2
5
10
```

**check the no is prime or not**

```
n=eval(input("enter a number"))
for i in range(2,n):
 if(n%i==0):
```

```

print("The num is not a prime")
break
else:
 print("The num is a prime number.")

```

### output

```

enter a no:7
The num is a prime number.

```

### check a number is perfect number or not

```

n=eval(input("enter a number:"))
sum=0
for i in range(1,n,1):
 if(n%i==0):
 sum=sum+i
 if(sum==n):
 print("the number is perfect number")
 else:
 print("the number is not perfect number")

```

### Output

```

enter a number:6
the number is perfect number

```

## 7. LOOP CONTROL STRUCTURES

### BREAK

- Break statements can alter the flow of a loop.
- It terminates the current
- loop and executes the remaining statement outside the loop.
- If the loop has else statement, that will also gets terminated and come out of the loop completely.

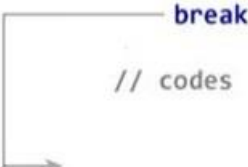
### Syntax:

Break

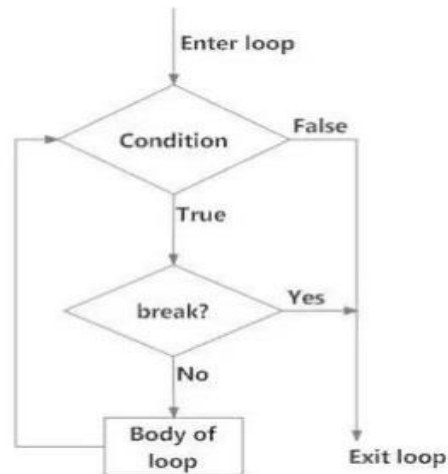
```

while (test Expression):
 // codes
 if (condition for break):
 break
 // codes

```



### Flowchart



### example

```

for i in "welcome":
 if(i=="c"):
 break
 print(i)

```

### Output

```

w
e
l

```

### CONTINUE

- It terminates the current iteration and transfer the control to the next iteration in the loop.

### Syntax:

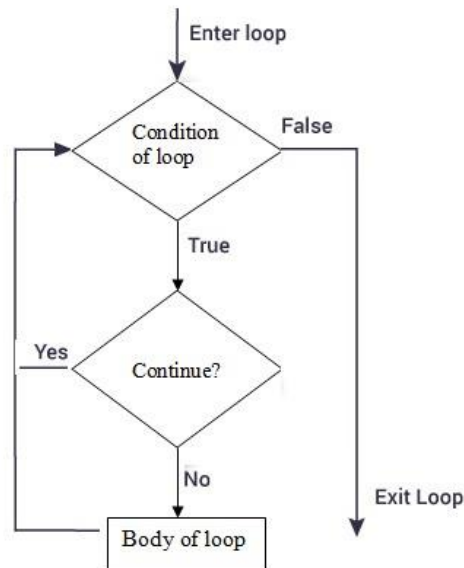
Continue

```

→ while (test Expression):
 // codes
 if (condition for continue):
 continue
 // codes

```

### Flowchart



### Example:

```

for i in "welcome":
 if(i=="c"):
 continue
 print(i)

```

### Output

```

w
e
l
o
m
e

```

### PASS

- It is used when a statement is required syntactically but you don't want any code to execute.
- It is a null statement, nothing happens when it is executed.

### Syntax:

```

pass
break

```

### Example

```

for i in "welcome":
 if (i == "c"):
 pass
 print(i)

```

### Output

```

w

```

e  
l  
c  
o  
m  
e

### Difference between break and continue

| <b>break</b>                                                                          | <b>continue</b>                                                                                 |
|---------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| It terminates the current loop and executes the remaining statement outside the loop. | It terminates the current iteration and transfer the control to the next iteration in the loop. |
| syntax:<br>break                                                                      | syntax:<br>continue                                                                             |
| for i in "welcome":<br>if(i=="c"):<br>break<br>print(i)                               | for i in "welcome":<br>if(i=="c"):<br>continue<br>print(i)                                      |
| w<br>e<br>l                                                                           | w<br>e<br>l<br>o<br>m<br>e                                                                      |

### else statement in loops:

#### else in for loop:

- If else statement is used in for loop, the else statement is executed when the loop has reached the limit.
- The statements inside for loop and statements inside else will also execute.

#### example

```
for i in range(1,6):
 print(i)
else:
 print("the number greater than 6")
```

#### output

```
1
2
3
4
5 the number greater than 6
```

### else in while loop:

- If else statement is used within while loop , the else part will be executed when the condition become false.
- The statements inside for loop and statements inside else will also execute.

#### Program

```
i=1
while(i<=5):
 print(i)
 i=i+1
```

else:

```
print("the number greater than 5")
```

**output**

```
1
2
3
4
5
the number greater than 5
```

## 8. FUNCTIONS:

1. What is recursion? (or) Define Recursion with an example. (2 MARKS) (JAN2019)
2. Explain global and local scope. (or) Comment with an example on the use of local and global variable with the same identifier name. (2 MARKS) (MAY 2019)
3. Define recursive function. (2 MARKS) (JAN 2022)
4. Write a python program using function to find the sum of first n odd numbers and print result. (4 MARKS) (JAN 2022)

- *Function is a sub program which consists of set of instructions used to perform a specific task. A large program is divided into basic building blocks called function.*

### Need For Function:

- When the program is too complex and large they are divided into parts. Each part is separately coded and combined into single program. Each subprogram is called as function.
- Debugging, Testing and maintenance becomes easy when the program is divided into subprograms.
- Functions are used to avoid rewriting same code again and again in a program.
- Function provides code re-usability
- The length of the program is reduced.

### Types of function:

Functions can be classified into two categories:

- i) user defined function
- ii) Built in function

#### **i) Built in functions**

- Built in functions are the functions that are already created and stored in python.
- These built in functions are always available for usage and accessed by a programmer. It cannot be modified.

| Built in function                               | Description                                      |
|-------------------------------------------------|--------------------------------------------------|
| >>>max(3,4)<br>4                                | # returns largest element                        |
| >>>min(3,4)<br>3                                | # returns smallest element                       |
| >>>len("hello")<br>5                            | #returns length of an object                     |
| >>>range(2,8,1)<br>[2, 3, 4, 5, 6, 7]           | #returns range of given values                   |
| >>>round(7.8)<br>8.0                            | #returns rounded integer of the given number     |
| >>>chr(5)<br>\x05'                              | #returns a character (a string) from an integer  |
| >>>float(5)<br>5.0                              | #returns float number from string or integer     |
| >>>int(5.0)<br>5                                | # returns integer from string or float           |
| >>>pow(3,5)<br>243                              | #returns power of given number                   |
| >>>type( 5.6)<br><type 'float'>                 | #returns data type of object to which it belongs |
| >>>t=tuple([4,6.0,7])<br>(4, 6.0, 7)            | # to create tuple of items from list             |
| >>>print("good morning")<br>Good morning        | # displays the given object                      |
| >>>input("enter name: ")<br>enter name : George | # reads and returns the given string             |

## ii)User Defined Functions:

- User defined functions are the functions that programmers create for their requirement and use.
- These functions can then be **combined to form module** which can be used in other programs by importing them.
- Advantages of user defined functions:
  - Programmers working on large project can divide the workload by making different functions.
  - If repeated code occurs in a program, function can be used to include those codes and execute when needed by calling that function.

### **Function definition: (Sub program)**

- def keyword is used to define a function.
- Give the function name after def keyword followed by parentheses in which arguments are given.
- End with colon (:)
- Inside the function add the program statements to be executed
- End with or without return statement

### **Syntax:**

```
def fun_name(Parameter1,Parameter2...Parameter n):
 statement1
```

```
statement2...
statement n
return[expression]
```

**Example:**

```
def my_add(a,b):
 c=a+b
 return c
```

**Function Calling: (Main Function)**

- Once we have defined a function, we can call it from another function, program or even the Python prompt.
- To call a function we **simply type the function name with appropriate arguments.**

Example:

```
x=5
y=4
my_add(x,y)
```

**Flow of Execution:**

- The order in which statements are executed is called the **flow of execution**
- Execution always begins at the first statement of the program.
- Statements are executed one at a time, in order, from top to bottom.
- Function definitions do not alter the flow of execution of the program, but remember that statements inside the function are not executed until the function is called.
- Function calls are like a bypass in the flow of execution. Instead of going to the next statement, the flow jumps to the first line of the called function, executes all the statements there, and then comes back to pick up where it left off.

**Note:** When you read a program, don't read from top to bottom. Instead, follow the flow of execution. This means that you will read the **def** statements as you are scanning from top to bottom, but you should skip the statements of the function definition until you reach a point where that function is called.

**Function Prototypes:**

- i. Function without arguments and without return type
- ii. Function with arguments and without return type
- iii. Function without arguments and with return type
- iv. Function with arguments and with return type

**i) Function without arguments and without return type**

- In this type no argument is passed through the function call and no output is return to main function
- The sub function will read the input values perform the operation and print the result in the same block

**ii) Function with arguments and without return type**

- Arguments are passed through the function call but output is not return to the main function

### iii) Function without arguments and with return type

- In this type no argument is passed through the function call but output is return to the main function.

### iv) Function with arguments and with return type

- In this type arguments are passed through the function call and output is return to the main function

## Without Return Type

### Without argument

```
def add():
 a=int(input("enter a"))
 b=int(input("enter b"))
 c=a+b
 print(c)

add()
```

### OUTPUT:

```
enter a 5
enter b 10
15
```

### With argument

```
def add(a,b):
 c=a+b
 print(c)
a=int(input("enter a"))
b=int(input("enter b"))
add(a,b)
```

### OUTPUT:

```
enter a 5
enter b 10
15
```

## With return type

### Without argument

```
def add():
 a=int(input("enter a"))
 b=int(input("enter b"))
 c=a+b
 return c

c=add()
print(c)
```

### OUTPUT:

```
enter a 5
enter b 10
```

15

### With argument

```
def add(a,b):
 c=a+b
 return c
a=int(input("enter a"))
b=int(input("enter b"))
c=add(a,b)
print(c)
```

### OUTPUT:

```
enter a 5
enter b 10
15
```

## Parameters And Arguments:

### Parameters:

- Parameters are the value(s) provided in the parenthesis when we write function header.
- These are the values required by function to work.
- If there is more than one value required, all of them will be listed in parameter list separated by **comma**.
- Example: def my\_add(a,b):

### Arguments :

- Arguments are the value(s) provided in function call/invoke statement.
- List of arguments should be supplied in same way as parameters are listed.
- Bounding of parameters to arguments is done 1:1, and so there should be same number and type of arguments as mentioned in parameter list.

Example: my\_add(x,y)

### RETURN STATEMENT:

- The **return statement is used to exit a function** and go back to the place from where it was called.
- If the return statement has no arguments, then it will not return any values. But exits from function.

### Syntax:

```
return[expression]
```

### Example:

```
def my_add(a,b):
 c=a+b
 return c
x=5
y=4
print(my_add(x,y))
```

**Output:**

9

### **ARGUMENTS TYPES:**

1. Required Arguments
2. Keyword Arguments
3. Default Arguments
4. Variable length Arguments

#### **1. Required Arguments:**

- The number of arguments in the function call should match exactly with the function definition.

```
def my_details(name, age):
 print("Name: ", name)
 print("Age ", age)
 return
my_details("george",56)
```

**Output:**

Name: george  
Age 56

#### **2. Keyword Arguments:**

- Python interpreter is able to use the keywords provided to match the values with parameters even though if they are arranged in out of order.

```
def my_details(name, age):
 print("Name: ", name)
 print("Age ", age)
 return
my_details(age=56,name="george")
```

**Output:**

Name: george  
Age 56

#### **3. Default Arguments:**

- Assumes a default value if a value is not provided in the function call for that argument.

```
def my_details(name, age=40):
 print("Name: ", name)
 print("Age ", age)
 return
my_details(name="george")
```

**Output:**

Name: george  
Age 40

#### **4. Variable length Arguments**

- If we want to specify more arguments than specified while defining the function, variable length arguments are used. It is denoted by \*symbol before parameter.

```
def my_details(*name):
 print(*name)
my_details("rajan","rahul","micheal", "ärjun")
```

**Output:**

rajan rahul micheal ärjun

### **Fruitful function:**

A function that returns a value is called fruitful function.

### **Example:**

Root=sqrt(25)

### **Example:**

```
def add():
 a=10
 b=20
 c=a+b
 return c
c=add()
print(c)
```

### **Void Function**

- A function that perform action but don't return any value.

### **Example:**

print("Hello")

### **Example:**

```
def add():
 a=10
 b=20
 c=a+b
 print(c)
add()
```

### **Return values:**

- return keywords are used to return the values from the function.

### **example:**

```
return a – return 1 variable
return a,b– return 2 variables
return a,b,c– return 3 variables
return a+b– return expression
return 8– return value
```

### **PARAMETERS / ARGUMENTS:**

- Parameters are the variables which used in the function definition. Parameters are inputs to functions. Parameter receives the input from the function call.
- It is possible to define more than one parameter in the function definition.

### Local and Global Scope

#### Global Scope

- The *scope* of a variable refers to the places that you can see or access a variable.
- A variable with global scope can be used anywhere in the program.
- It can be created by defining a variable outside the function.

| Example                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | output                                                                                                                                                                  |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> a=50 def add():     b=20     c=a+b     print(c) def sub():     b=30     c=a-b     print(c) print(a)           </pre> <div style="display: flex; justify-content: space-around; margin-top: 10px;"> <div style="text-align: center;">  <div style="border: 1px solid black; padding: 2px; display: inline-block;">Global Variable</div> </div> <div style="text-align: center;">  <div style="border: 1px solid black; padding: 2px; display: inline-block;">Local Variable</div> </div> </div> | <div style="text-align: center; margin-bottom: 20px;">70</div> <div style="text-align: center; margin-bottom: 20px;">20</div> <div style="text-align: center;">50</div> |

**Local Scope** A variable with local scope can be used only within the function .

| Example                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | output                                                                                                                                                                                                                  |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> def add():     b=20     c=a+b     print(c) def sub():     b=30     c=a-b     print(c) print(a) print(b)           </pre> <div style="display: flex; justify-content: space-around; margin-top: 10px;"> <div style="text-align: center;">  <div style="border: 1px solid black; padding: 2px; display: inline-block;">Local Variable</div> </div> <div style="text-align: center;">  <div style="border: 1px solid black; padding: 2px; display: inline-block;">Local Variable</div> </div> </div> | <div style="text-align: center; margin-bottom: 20px;">70</div> <div style="text-align: center; margin-bottom: 20px;">20</div> <div style="text-align: center;">error</div> <div style="text-align: center;">error</div> |

#### **Function Composition:**

- Function Composition is the ability to call one function from within another function
- It is a way of combining functions such that the result of each function is passed as the argument of the next function.
- In other words the output of one function is given as the input of another function is known as function composition.

#### **Example:**

```

math.sqrt(math.log(10))
def add(a,b):
 c=a+b

```

```
 return c
def mul(c,d):
 e=c*d
 return e
c=add(10,20)
e=mul(c,30)
print(e)
```

**Output:**

900

**find sum and average using function composition**

```
def sum(a,b):
 sum=a+b
 return sum
def avg(sum):
 avg=sum/2
 return avg
a=eval(input("enter a:"))
b=eval(input("enter b:"))
sum=sum(a,b)
avg=avg(sum)
print("the avg is",avg)
```

**output**

enter a:4  
enter b:8  
the avg is 6.0

**Recursion**

- A function calling itself till it reaches the base value - stop point of function call.  
Example: factorial of a given number using recursion

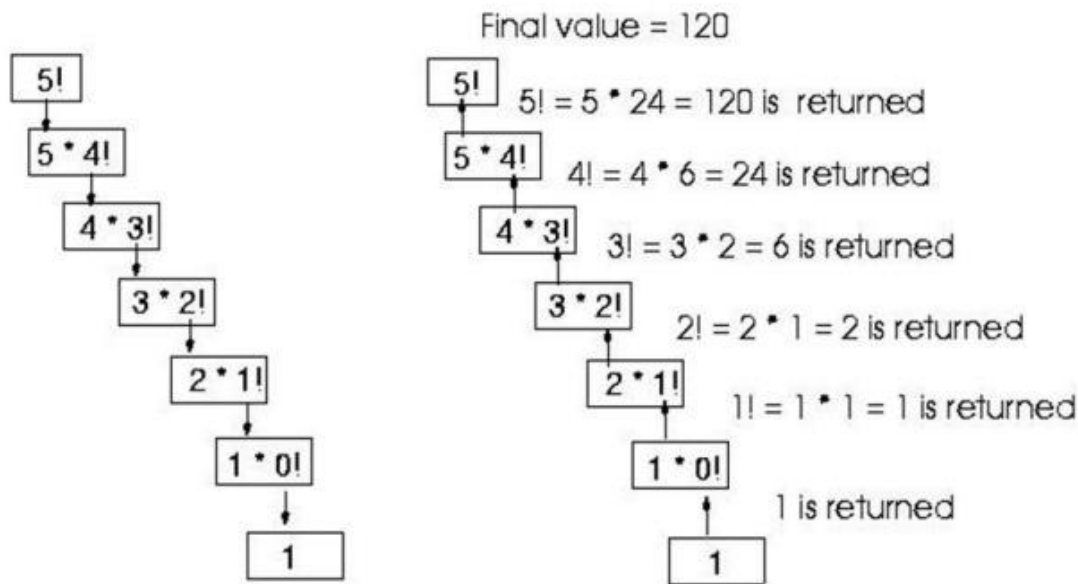
**Factorial of n**

```
def fact(n):
 if(n==1):
 return 1
 else:
 return n*fact(n-1)
n=eval(input("enter no. to find fact:"))
fact=fact(n)
print("Fact is",fact)
```

**Output**

enter no. to find fact:5  
Fact is 120

**Explanation**



### Examples:

1. sum of n numbers using recursion
2. exponential of a number using recursion

### Sum of n numbers

```
def sum(n):
 if(n==1):
 return 1
 else:
 return n*sum(n-1)
n=eval(input("enter no. to find sum:"))
sum=sum(n)
print("Fact is",sum)
```

### Output

```
enter no. to find sum:10
Fact is 55
```

## THE ANONYMOUS FUNCTION – LAMBDA

Lambda is the keyword used to create small functions. It lets to define one – line mini functions on the fly. Python borrowed concept of lambda functions from lisp. Lambda functions can have any number of arguments but only one expression. The expression is evaluated and returned. It can be used wherever functions objects are required. An anonymous function cannot be a direct call to print because lambda requires an expression. Lambda functions have their own local namespace and cannot access variable other than those in their parameter list and those in global namespace.

### Syntax:

lambda [arg1[arg2,...argn]] : expression

### Program 1

```
sum = lambda arg1, arg 2: arg1 + arg2
print("total:",sum (100,200))
```

#### Output:

Total: 300

### Program 2

```
double = lambda n : n*2
print("calling
lambda fn:",double(10))
```

#### Output:

Calling lambda fn : 20

### Program 3: lambda within lambda

```
add-two=lambda n : n+2
multiply-add-two=lambda n : add-two(n*2)
print ("lambda:",add-two(3))
print("lambda within lambda:",multiply-add-two(3))
```

#### Output:

lambda: 5  
lambda within lambda: 8

### Use of lambda functions

- Lambda functions can be used when we require a nameless function for a short period of time.
- Lambda functions are used along with built-in functions like filter(), map(),etc.

### Lambda function with filter ():

- The filter () function in python takes in a function and a list as arguments. The function is called for all the items in the list and a new list is returned which contain items for which the function evaluates to true.
- The following example uses filter () fn to filter out only even numbers from a list.

### Program

```
old-list=[21,25,24,27,30,28,32,33,34]
new-list=list(filter(lambda no : (no%2==0),old-list))
print (new-list)
```

#### Output:

[24,30,28,32,34]

### Lambda function with map ():

- The map () function in python takes in a function and a list. The function is called with all the items in a list and a new list is returned which contains values provided by the functions for each item.
- The following examples uses map () functions to double all the items in the list.

## Program

```
old-list=[2,5,8,3,10,7]
new-list=[list(map(lambda no : no*2, old-list))]
print (new-list)
```

## Output:

```
[4,10,16,6,20,14]
```

## 9. STRINGS:

1. Write a python code to search a string in the given list. (16 MARKS) (MAY 2018)
2. Write a python program to generate all permutations of a given string using built-in function. (8 MARKS) (DEC 2019)
3. Write a program to display a set of strings using range () function. (2 MARKS)(NOV 2019)
4. Discuss in detail about string slices and string immutability. (16 MARKS) (JAN 2019)
5. Analyse String slicing. Illustrate how it is done in python with example. (16 MARKS) (MAY 2019)

- Strings
- String slices
- Immutability
- String functions and methods
- String module

### Strings:

- String is defined as sequence of characters represented in quotation marks<sup>2</sup>(either single quotes ( ' ') or double quotes ( " ")).
- An individual character in a string is accessed using a index.
- The index should always be an integer (positive or negative).
- A index starts from 0 to n-1.
- Strings are immutable i.e. the contents of the string cannot be changed after it is created.
- Python will get the input at run time by default as a string.
- Python does not support character data type. A string of size 1 can be treated as characters.

1. single quotes ( ' ' )
2. double quotes ( " " )
3. triple quotes ( " " " " " " )

### Operations on string:

1. Indexing
2. Slicing
3. Concatenation
4. Repetitions
5. Member ship

| String A       | H  | E  | L  | L  | O  |
|----------------|----|----|----|----|----|
| Positive Index | 0  | 1  | 2  | 3  | 4  |
| Negative Index | -5 | -4 | -3 | -2 | -1 |

|                      |                                                                                                               |                                                                                                                                                                                                  |
|----------------------|---------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>indexing</b>      | <pre>&gt;&gt;&gt;a="HELLO" &gt;&gt;&gt;print(a[0]) &gt;&gt;&gt;H &gt;&gt;&gt;print(a[-1]) &gt;&gt;&gt;0</pre> | <ul style="list-style-type: none"> <li>❖ Positive indexing helps in accessing the string from the beginning</li> <li>❖ Negative subscript helps in accessing the string from the end.</li> </ul> |
| <b>Slicing:</b>      | <pre>Print[0:4] – HELL Print[:3] – HEL Print[0: ]- HELLO</pre>                                                | <p>The Slice[start : stop] operator extracts sub string from the strings.<br/>A segment of a string is called a slice.</p>                                                                       |
| <b>Concatenation</b> | <pre>a="save" b="earth" &gt;&gt;&gt;print(a+b) saveearth</pre>                                                | <p>The + operator joins the text on both sides of the operator.</p>                                                                                                                              |
| <b>Repetitions:</b>  | <pre>a:"RIT CHENNAI" &gt;&gt;&gt;print(3*a) RIT CHENNAIRITCHENNAI RIT CHENNAI</pre>                           | <p>The * operator repeats the string on the left hand side times the value on right hand side.</p>                                                                                               |
| <b>Membership:</b>   | <pre>&gt;&gt;&gt; s="good morning" &gt;&gt;&gt;"m" in s True &gt;&gt;&gt; "a" not in s True</pre>             | <p>Using membership operators to check a particular character is in string or not. Returns true if present</p>                                                                                   |

### String slices:

- A part of a string is called string slices.
- **The process of extracting a sub string from a string is called slicing.**

|                                     |                                                                |                                                                                                                     |
|-------------------------------------|----------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| <b>Slicing:</b><br><b>a="HELLO"</b> | <pre>Print[0:4] – HELL Print[:3] – HEL Print[0: ]- HELLO</pre> | <p>The Slice[n : m] operator extracts sub string from the strings.<br/>A segment of a string is called a slice.</p> |
|-------------------------------------|----------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|

### Immutability:

- Python strings are "immutable" as they cannot be changed after they are created.
- Therefore [ ] operator cannot be used on the left side of an assignment.

| operations         | Example                | output                                                      |
|--------------------|------------------------|-------------------------------------------------------------|
| element assignment | a="PYTHON"<br>a[0]='x' | TypeError: 'str' object does not support element assignment |
| element deletion   | a="PYTHON"<br>del a[0] | TypeError: 'str' object doesn't support element deletion    |
| delete a string    | a="PYTHON"<br>del a    | NameError: name 'my_string' is not defined                  |

### string built in functions and methods:

- A **method** is a function that “belongs to” an object.

### Syntax to access the method

**Stringname.method()**

a="happy birthday"

here, a is the string name.

|    | syntax                     | example                                                                  | description                                                                                |
|----|----------------------------|--------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| 1  | a.capitalize()             | >>> a.capitalize()<br>' Happy birthday'                                  | capitalize only the first letter in a string                                               |
| 2  | a.upper()                  | >>> a.upper()<br>'HAPPY BIRTHDAY'                                        | change string to upper case                                                                |
| 3  | a.lower()                  | >>> a.lower()<br>' happy birthday'                                       | change string to lower case                                                                |
| 4  | a.title()                  | >>> a.title()<br>' Happy Birthday '                                      | change string to title case i.e. first characters of all the words are capitalized.        |
| 5  | a.swapcase()               | >>> a.swapcase()<br>'HAPPY BIRTHDAY'                                     | change lowercase characters to uppercase and vice versa                                    |
| 6  | a.split()                  | >>> a.split()<br>['happy', 'birthday']                                   | returns a list of words separated by space                                                 |
| 7  | a.center(width,"fillchar") | >>>a.center(19,"*")<br>***happy birthday***                              | pads the string with the specified “fillchar” till the length is equal to “width”          |
| 8  | a.count(substring)         | >>> a.count('happy')<br>1                                                | returns the number of occurrences of substring                                             |
| 9  | a.replace(old,new)         | >>>a.replace('happy',<br>'wishyou happy')<br>'wishyou happy<br>birthday' | replace all old substrings with new substrings                                             |
| 10 | a.join(b)                  | >>> b="happy"<br>>>> a="-"<br>>>> a.join(b)<br>'h-a-p-p-y'               | returns a string concatenated with the elements of an iterable. (Here “a” is the iterable) |

|    |                         |                               |                                                                                     |
|----|-------------------------|-------------------------------|-------------------------------------------------------------------------------------|
| 11 | a.isupper()             | >>> a.isupper()<br>False      | checks whether all the case-based characters (letters) of the string are uppercase. |
| 12 | a.islower()             | >>> a.islower()<br>True       | checks whether all the case-based characters (letters) of the string are lowercase. |
| 13 | a.isalpha()             | >>> a.isalpha()<br>False      | checks whether the string consists of alphabetic characters only.                   |
| 14 | a.isalnum()             | >>> a.isalnum()<br>False      | checks whether the string consists of alphanumeric characters.                      |
| 15 | a.isdigit()             | >>> a.isdigit()<br>False      | checks whether the string consists of digits only.                                  |
| 16 | a.isspace()             | >>> a.isspace()<br>False      | checks whether the string consists of whitespace only.                              |
| 17 | a.istitle()             | >>> a.istitle()<br>False      | checks whether string is title cased.                                               |
| 18 | a.startswith(substring) | >>> a.startswith("h")<br>True | checks whether string starts with substring                                         |
| 19 | a.endswith(substring)   | >>> a.endswith("y")<br>True   | checks whether the string ends with the substring                                   |
| 20 | a.find(substring)       | >>> a.find("happy")<br>0      | returns index of substring, if it is found. Otherwise -1 is returned.               |
| 21 | len(a)                  | >>> len(a)<br>>>> 14          | Return the length of the string                                                     |
| 22 | min(a)                  | >>> min(a)<br>>>> ' '         | Return the minimum character in the string                                          |
| 23 | max(a)                  | max(a)<br>>>> 'y'             | Return the maximum character in the string                                          |

### **String modules:**

- A module is a file containing Python definitions, functions, statements.
- Standard library of Python is extended as modules.
- To use these modules in a program, programmer needs to import the module.
- Once we import a module, we can reference or use to any of its functions or variables in our code.
- There is large number of standard modules also available in python.
- Standard modules can be imported the same way as we import our user-defined modules.

### **Syntax:**

```
import module_name
```

### **Example**

```
import string
print(string.punctuation)
print(string.digits)
print(string.printable)
```

```
print(string.capwords("happy birthday"))
print(string.hexdigits)
print(string.octdigits)
```

### output

```
!"#$%&'()*+,-./:;<=>?@[\]^_`{|}~
0123456789
0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHJIJ
KLMNOPQRSTUVWXYZ!"#$%&'()*+,-
./:;<=>?@[\]^_`{|}~
Happy Birthday
0123456789abcdefABCDEF
01234567
```

### Escape sequences in string

| Escape Sequence | Description             | example                                  |
|-----------------|-------------------------|------------------------------------------|
| \n              | new line                | >>> print("hai \nhello")<br>hai<br>hello |
| \\              | prints Backslash (\)    | >>> print("hai\\hello")<br>hai\hello     |
| \'              | prints Single quote (') | >>> print("'")<br>'                      |
| \"              | prints Double quote (") | >>>print("\"")<br>"                      |
| \t              | prints tab sapace       | >>>print("hai\thello")<br>hai   hello    |
| \a              | ASCII Bell (BEL)        | >>>print("\a")                           |

## 10.LISTS

1. What is a list? (2 MARKS) (JAN 2018)
2. How list differs from tuple. (2 MARKS) (JAN 2018)
3. How to slice a list in Python? (2 MARKS) (NOV 2018)
4. Relate String and List? (2 MARKS) (JAN 2019)
5. How to create a list in python? Illustrate the use of negative indexing of list with example. (8 MARKS) (MAY 2019)
6. How will you update list items? Give one example. (8 MARKS) (DEC 2019)
7. Explain in detail about lists, list operations and list slices. (8 MARKS) (JAN 2019)
8. In python how the values stored in list are accessed? Should the elements of a list be of a same datatype? (2 MARKS) (JAN 2022)

- List is an ordered sequence of items. Values in the list are called elements / items.
- It can be written as a list of comma-separated items (values) between **square brackets[ ]**.
- Items in the lists can be of different data types.

### Operations on list:

1. Indexing
2. Slicing
3. Concatenation
4. Repetitions
5. Updating
6. Membership
7. Comparison

| operations           | examples                                                                                                                          | description                                                                                                                       |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| <b>create a list</b> | <pre>&gt;&gt;&gt; a=[2,3,4,5,6,7,8,9,10] &gt;&gt;&gt; print(a) [2, 3, 4, 5, 6, 7, 8, 9, 10]</pre>                                 | in this way we can create a list at compile time                                                                                  |
| <b>Indexing</b>      | <pre>&gt;&gt;&gt; print(a[0]) 2 &gt;&gt;&gt; print(a[8]) 10 &gt;&gt;&gt; print(a[-1]) 10</pre>                                    | Accessing the item in the position 0<br>Accessing the item in the position 8<br>Accessing a last element using negative indexing. |
| <b>Slicing</b>       | <pre>&gt;&gt;&gt; print(a[0:3]) [2, 3, 4] &gt;&gt;&gt; print(a[0:]) [2, 3, 4, 5, 6, 7, 8, 9, 10]</pre>                            | Printing a part of the list.                                                                                                      |
| <b>Concatenation</b> | <pre>&gt;&gt;&gt;b=[20,30] &gt;&gt;&gt; print(a+b) [2, 3, 4, 5, 6, 7, 8, 9, 10, 20, 30]</pre>                                     | Adding and printing the items of two lists.                                                                                       |
| <b>Repetition</b>    | <pre>&gt;&gt;&gt; print(b*3) [20, 30, 20, 30, 20, 30]</pre>                                                                       | Create a multiple copies of the same list.                                                                                        |
| <b>Updating</b>      | <pre>&gt;&gt;&gt; print(a[2]) 4 &gt;&gt;&gt; a[2]=100 &gt;&gt;&gt; print(a) [2, 3, 100, 5, 6, 7, 8, 9, 10]</pre>                  | Updating the list using index value.                                                                                              |
| <b>Membership</b>    | <pre>&gt;&gt;&gt; a=[2,3,4,5,6,7,8,9,10] &gt;&gt;&gt; 5 in a True &gt;&gt;&gt; 100 in a False &gt;&gt;&gt; 2 not in a False</pre> | Returns True if element is present in list. Otherwise returns false.                                                              |
| <b>Comparison</b>    | <pre>&gt;&gt;&gt; a=[2,3,4,5,6,7,8,9,10] &gt;&gt;&gt;b=[2,3,4] &gt;&gt;&gt; a==b False &gt;&gt;&gt; a!=b True</pre>               | Returns True if all elements in both elements are same. Otherwise returns false                                                   |

### List slices:

- List slicing is an operation that extracts a subset of elements from an list and packages them as another list.

### Syntax:

**Listname[start:stop]**

**Listname[start:stop:steps]**

- default start value is 0
- default stop value is n-1
- [:] this will print the entire list
- [2:2] this will create a empty slice

| slices          | example                                        | description                                           |
|-----------------|------------------------------------------------|-------------------------------------------------------|
| <b>a[0:3]</b>   | >>> a=[9,8,7,6,5,4]<br>>>> a[0:3]<br>[9, 8, 7] | Printing a part of a list from 0 to 2.                |
| <b>a[:4]</b>    | >>> a[:4]<br>[9, 8, 7, 6]                      | Default start value is 0. so prints from 0 to 3       |
| <b>a[1:]</b>    | >>> a[1:]<br>[8, 7, 6, 5, 4]                   | default stop value will be n-1. so prints from 1 to 5 |
| <b>a[:]</b>     | >>> a[:]<br>[9, 8, 7, 6, 5, 4]                 | Prints the entire list.                               |
| <b>a[2:2]</b>   | >>> a[2:2]<br>[]                               | print an empty slice                                  |
| <b>a[0:6:2]</b> | >>> a[0:6:2]<br>[9, 7, 5]                      | Slicing list values with step size 2.                 |
| <b>a[::-1]</b>  | >>> a[::-1]<br>[4, 5, 6, 7, 8, 9]              | Returns reverse of given list values                  |

### List methods:

- Methods used in lists are used to manipulate the data quickly.
- These methods work only on lists.
- They do not work on the other sequence types that are not mutable, that is, the values they contain cannot be changed, added, or deleted.

### syntax:

**list name.method name( element/index/list)**

|    | <b>syntax</b>                        | <b>example</b>                                                                                                 | <b>description</b>                                         |
|----|--------------------------------------|----------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|
| 1  | <code>a.append(element)</code>       | <pre>&gt;&gt;&gt; a=[1,2,3,4,5] &gt;&gt;&gt; a.append(6) &gt;&gt;&gt; print(a) [1, 2, 3, 4, 5, 6]</pre>        | Add an element to the end of the list                      |
| 2  | <code>a.insert(index,element)</code> | <pre>&gt;&gt;&gt; a.insert(0,0) &gt;&gt;&gt; print(a) [0, 1, 2, 3, 4, 5, 6]</pre>                              | Insert an item at the defined index                        |
| 3  | <code>a.extend(b)</code>             | <pre>&gt;&gt;&gt; b=[7,8,9] &gt;&gt;&gt; a.extend(b) &gt;&gt;&gt; print(a) [0, 1, 2, 3, 4, 5, 6, 7, 8,9]</pre> | Add all elements of a list to the another list             |
| 4  | <code>a.index(element)</code>        | <pre>&gt;&gt;&gt; a.index(8) 8</pre>                                                                           | Returns the index of the first matched item                |
| 5  | <code>a.sort()</code>                | <pre>&gt;&gt;&gt; a.sort() &gt;&gt;&gt; print(a) [0, 1, 2, 3, 4, 5, 6, 7, 8]</pre>                             | Sort items in a list in ascending order                    |
| 6  | <code>a.reverse()</code>             | <pre>&gt;&gt;&gt; a.reverse() &gt;&gt;&gt; print(a) [8, 7, 6, 5, 4, 3, 2, 1, 0]</pre>                          | Reverse the order of items in the list                     |
| 7  | <code>a.pop()</code>                 | <pre>&gt;&gt;&gt; a.pop() 0</pre>                                                                              | Removes and returns an element at the last element         |
| 8  | <code>a.pop(index)</code>            | <pre>&gt;&gt;&gt; a.pop(0) 8</pre>                                                                             | Remove the particular element and return it.               |
| 9  | <code>a.remove(element)</code>       | <pre>&gt;&gt;&gt; a.remove(1) &gt;&gt;&gt; print(a) [7, 6, 5, 4, 3, 2]</pre>                                   | Removes an item from the list                              |
| 10 | <code>a.count(element)</code>        | <pre>&gt;&gt;&gt; a.count(6) 1</pre>                                                                           | Returns the count of number of items passed as an argument |
| 11 | <code>a.copy()</code>                | <pre>&gt;&gt;&gt; b=a.copy() &gt;&gt;&gt; print(b) [7, 6, 5, 4, 3, 2]</pre>                                    | Returns a shallow copy of the list                         |
| 12 | <code>len(list)</code>               | <pre>&gt;&gt;&gt; len(a) 6</pre>                                                                               | return the length of the length                            |
| 13 | <code>min(list)</code>               | <pre>&gt;&gt;&gt; min(a) 2</pre>                                                                               | return the minimum element in a list                       |
| 14 | <code>max(list)</code>               | <pre>&gt;&gt;&gt; max(a) 7</pre>                                                                               | return the maximum element in a list.                      |
| 15 | <code>a.clear()</code>               | <pre>&gt;&gt;&gt; a.clear() &gt;&gt;&gt; print(a) []</pre>                                                     | Removes all items from the list.                           |
| 16 | <code>del(a)</code>                  | <pre>&gt;&gt;&gt; del(a) &gt;&gt;&gt; print(a) Error: name 'a' is not defined</pre>                            | delete the entire list.                                    |

## List loops:

1. For loop
2. While loop
3. Infinite loop

### 1. List using For Loop:

- The for loop in Python is used to iterate over a sequence (list, tuple, string) or other iterable objects.
- Iterating over a sequence is called traversal.
- Loop continues until we reach the last item in the sequence.
- The body of for loop is separated from the rest of the code **using indentation**.

#### Syntax:

**for val in sequence:**

| Accessing element                                                | output                     |
|------------------------------------------------------------------|----------------------------|
| a=[10,20,30,40,50]<br>for i in a:<br>print(i)                    | 1<br>2<br>3<br>4<br>5      |
| Accessing index                                                  | output                     |
| a=[10,20,30,40,50]<br>for i in range(0,len(a),1):<br>print(i)    | 0<br>1<br>2<br>3<br>4      |
| Accessing element using range:                                   | output                     |
| a=[10,20,30,40,50]<br>for i in range(0,len(a),1):<br>print(a[i]) | 10<br>20<br>30<br>40<br>50 |

### **2. List using While loop**

- The while loop in Python is used to iterate over a block of code as long as the test expression (condition) is true.
- When the condition is tested and the result is false, the loop body will be skipped and the first statement after the while loop will be executed.

#### Syntax:

**while (condition):**  
    **body of while**

### Sum of elements in list

```
a=[1,2,3,4,5]
i=0
sum=0
while i<len(a):
 sum=sum+a[i]
 i=i+1
print(sum)
```

#### Output:

15

### 3. Infinite Loop

- A loop becomes infinite loop if the condition given never becomes false. It keeps on running. Such loops are called infinite loop.

#### Example

```
a=1
while (a==1):
 n=int(input("enter the number"))
 print("you entered:" , n)
```

#### Output:

```
Enter the number 10
you entered:10
Enter the number 12
you entered:12
Enter the number 16
you entered:16
```

#### Mutability:

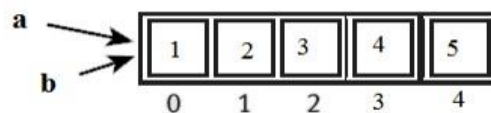
- Lists are mutable. (can be changed)
- Mutability is the ability for certain types of data to be changed without entirely recreating it.
- An item can be changed in a list by accessing it directly as part of the assignment statement.
- Using the indexing operator (square brackets[ ]) on the left side of an assignment, one of the list items can be updated.

| Example                                                                                                             | description                                                                                             |
|---------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| <pre>&gt;&gt;&gt; a=[1,2,3,4,5] &gt;&gt;&gt; a[0]=100 &gt;&gt;&gt; print(a) [100, 2, 3, 4, 5]</pre>                 | changing single element                                                                                 |
| <pre>&gt;&gt;&gt; a=[1,2,3,4,5] &gt;&gt;&gt; a[0:3]=[100,100,100] &gt;&gt;&gt; print(a) [100, 100, 100, 4, 5]</pre> | changing multiple element                                                                               |
| <pre>&gt;&gt;&gt; a=[1,2,3,4,5] &gt;&gt;&gt; a[0:3]=[ ] &gt;&gt;&gt; print(a) [4, 5]</pre>                          | The elements from a list can also be removed by assigning the empty list to them.                       |
| <pre>&gt;&gt;&gt; a=[1,2,3,4,5] &gt;&gt;&gt; a[0:0]=[20,30,45] &gt;&gt;&gt; print(a) [20,30,45,1, 2, 3, 4, 5]</pre> | The elements can be inserted into a list by squeezing them into an empty slice at the desired location. |

### Aliasing(copying):

- Creating a copy of a list is called aliasing. When you create a copy both list will be having same memory location. changes in one list will affect another list.
- **Alaising** refers to having different names for same list values.

| Example                                                                       | Output:                                                     |
|-------------------------------------------------------------------------------|-------------------------------------------------------------|
| <pre>a= [1, 2, 3 ,4 ,5] b=a print (b) a is b a[0]=100 print(a) print(b)</pre> | <pre>[1, 2, 3, 4, 5] True [100,2,3,4,5] [100,2,3,4,5]</pre> |



- In this a single list object is created and modified using the subscript operator.
- When the first element of the list named “a” is replaced, the first element of the list named “b” is also replaced.
- This type of change is what is known as a **side effect**. This happens because after the assignment **b=a**, the variables **a** and **b** refer to the exact same list object.
- They are **aliases** for the same object. This phenomenon is known as **aliasing**.
- To prevent aliasing, a new object can be created and the contents of the original can be copied which is called **cloning**.

### Clonning:

- To avoid the disadvantages of copying we are using cloning. creating a copy of a same list of elements with two different memory locations is called cloning.
- Changes in one list will not affect locations of another list.
- Cloning is a process of making a copy of the list without modifying the original list.

1. Slicing
2. list() method
3. copy() method

#### **clonning using Slicing**

```
>>>a=[1,2,3,4,5]
>>>b=a[:]
>>>print(b)
[1,2,3,4,5]
>>>a is b
False
```

#### **clonning using List( ) method**

```
>>>a=[1,2,3,4,5]
>>>b=list
>>>print(b)
[1,2,3,4,5]
>>>a is b
false
>>>a[0]=100
>>>print(a)
>>>a=[100,2,3,4,5]
>>>print(b)
>>>b=[1,2,3,4,5]
```

#### **clonning using copy() method**

```
a=[1,2,3,4,5]
>>>b=a.copy()
>>> print(b) [1, 2, 3, 4, 5]
>>> a is b
False
```

#### **List as parameters:**

- In python, arguments are passed by reference.
- If any changes are done in the parameter which refers within the function, then the changes also reflects back in the calling function.
- When a list to a function is passed, the function gets a reference to the list.
- Passing a list as an argument actually passes a reference to the list, not a copy of the list.
- Since lists are mutable, changes made to the elements referenced by the parameter change the same list that the argument is referencing.

#### **Example 1:**

```
def remove(a):
```

```
a.remove(1)
a=[1,2,3,4,5]
remove(a)
print(a)
```

**Output**

```
[2,3,4,5]
```

**Example 2:**

```
def inside(a):
 for i in range(0,len(a),1):
 a[i]=a[i]+10
 print("inside",a)
a=[1,2,3,4,5]
inside(a)
print("outside",a)
```

**Output**

```
inside [11, 12, 13, 14, 15]
outside [11, 12, 13, 14, 15]
```

**Example 3**

```
def insert(a):
 a.insert(0,30)
 a=[1,2,3,4,5]
insert(a)
print(a)
```

**output**

```
[30, 1, 2, 3, 4, 5]
```

**Advanced list processing:****List Comprehension:**

- List comprehensions provide a concise way to apply operations on a list.
- It creates a new list in which each element is the result of applying a given operation in a list.
- It consists of brackets containing an expression followed by a “for” clause, then a list.
- The list comprehension always returns a result list.

**Syntax**

**list=[ expression for item in list if conditional ]**

| List Comprehension                                                                                  | Output               |
|-----------------------------------------------------------------------------------------------------|----------------------|
| >>>L=[x**2 for x in range(0,5)]<br>>>>print(L)                                                      | [0, 1, 4, 9, 16]     |
| >>>[x for x in range(1,10) if x%2==0]                                                               | [2, 4, 6, 8]         |
| >>>[x for x in 'Python Programming' if x in ['a','e','i','o','u']]                                  | ['o', 'o', 'a', 'i'] |
| >>>mixed=[1,2,"a",3,4.2]<br>>>> [x**2 for x in mixed if type(x)==int]                               | [1, 4, 9]            |
| >>>[x+3 for x in [1,2,3]]                                                                           | [4, 5, 6]            |
| >>> [x*x for x in range(5)]                                                                         | [0, 1, 4, 9, 16]     |
| >>> num=[-1,2,-3,4,-5,6,-7]<br>>>> [x for x in num if x>=0]                                         | [2, 4, 6]            |
| >>> str=["this","is","an","example"]<br>>>> element=[word[0] for word in str]<br>>>> print(element) | ['t', 'i', 'a', 'e'] |

### Nested list:

- List inside another list is called nested list.

### **Example:**

```
>>> a=[56,34,5,[34,57]]
>>> a[0]
56
>>> a[3]
[34, 57]
>>> a[3][0]
34
>>> a[3][1]
57
```

## **11. TUPLE:**

1. What is tuple in python? How does it differ from list? (2 MARKS) (NOV 2019)
2. Write separate python program to illustrate create, access, concatenate and delete operations in a tuple. (8 MARKS)(JAN 2022)

- A tuple is same as list, except that the set of elements is enclosed in parentheses instead of square brackets.
- A tuple is an immutable list. i.e. once a tuple has been created, you can't add elements to a tuple or remove elements from the tuple.
- But tuple can be converted into list and list can be converted in to tuple.

| methods  | example                                                                                             | description                           |
|----------|-----------------------------------------------------------------------------------------------------|---------------------------------------|
| list( )  | <pre>&gt;&gt;&gt; a=(1,2,3,4,5) &gt;&gt;&gt; a=list(a) &gt;&gt;&gt; print(a) [1, 2, 3, 4, 5]</pre>  | it convert the given tuple into list. |
| tuple( ) | <pre>&gt;&gt;&gt; a=[1,2,3,4,5] &gt;&gt;&gt; a=tuple(a) &gt;&gt;&gt; print(a) (1, 2, 3, 4, 5)</pre> | it convert the given list into tuple. |

### Benefit of Tuple:

- Tuples are faster than lists.
- If the user wants to protect the data from accidental changes, tuple can be used.
- Tuples can be used as keys in dictionaries, while lists can't.

### Operations on Tuples:

1. Indexing
2. Slicing
3. Concatenation
4. Repetitions
5. Membership
6. Comparison

| Operations              | examples                                                                                                                          | description                                                                     |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| <b>Creating a tuple</b> | <pre>&gt;&gt;&gt;a=(20,40,60,"apple","ball")</pre>                                                                                | Creating the tuple with elements of different data types.                       |
| <b>Indexing</b>         | <pre>&gt;&gt;&gt;print(a[0]) 20 &gt;&gt;&gt; a[2] 60</pre>                                                                        | Accessing the item in the position 0<br>Accessing the item in the position 2    |
| <b>Slicing</b>          | <pre>&gt;&gt;&gt;print(a[1:3]) (40,60)</pre>                                                                                      | Displaying items from 1st till 2nd.                                             |
| <b>Concatenation</b>    | <pre>&gt;&gt;&gt; b=(2,4) &gt;&gt;&gt;print(a+b) &gt;&gt;&gt;(20,40,60,"apple","ball",2,4)</pre>                                  | Adding tuple elements at the end of another tuple elements                      |
| <b>Repetition</b>       | <pre>&gt;&gt;&gt;print(b*2) &gt;&gt;&gt;(2,4,2,4)</pre>                                                                           | repeating the tuple in n no of times                                            |
| <b>Membership</b>       | <pre>&gt;&gt;&gt; a=(2,3,4,5,6,7,8,9,10) &gt;&gt;&gt; 5 in a True &gt;&gt;&gt; 100 in a False &gt;&gt;&gt; 2 not in a False</pre> | Returns True if element is present in tuple. Otherwise returns false.           |
| <b>Comparison</b>       | <pre>&gt;&gt;&gt; a=(2,3,4,5,6,7,8,9,10) &gt;&gt;&gt;b=(2,3,4) &gt;&gt;&gt; a==b False &gt;&gt;&gt; a!=b True</pre>               | Returns True if all elements in both elements are same. Otherwise returns false |

## Tuple methods:

- Tuple is immutable so changes cannot be done on the elements of a tuple once it is assigned.

| methods        | example                                  | description                                  |
|----------------|------------------------------------------|----------------------------------------------|
| a.index(tuple) | >>> a=(1,2,3,4,5)<br>>>> a.index(5)<br>4 | Returns the index of the first matched item. |
| a.count(tuple) | >>> a=(1,2,3,4,5)<br>>>> a.count(3)<br>1 | Returns the count of the given element.      |
| len(tuple)     | >>> len(a)<br>5                          | return the length of the tuple               |
| min(tuple)     | >>> min(a)<br>1                          | return the minimum element in a tuple        |
| max(tuple)     | >>> max(a)<br>5                          | return the maximum element in a tuple        |
| del(tuple)     | >>> del(a)                               | Delete the entire tuple.                     |

## Tuple Assignment:

- Tuple assignment allows, variables on the left of an assignment operator and values of tuple on the right of the assignment operator.
- Multiple assignment works by creating a tuple of expressions from the right hand side, and a tuple of targets from the left, and then matching each expression to a target.
- Because multiple assignments use tuples to work, it is often termed tuple assignment.

## Uses of Tuple assignment:

- It is often useful to swap the values of two variables.

### Example:

#### Swapping using temporary variable:

```
a=20
b=50
temp = a
a = b
b = temp
print("value after swapping is",a,b)
```

#### Swapping using tuple assignment:

```
a=20
b=50
(a,b)=(b,a)
print("value after swapping is",a,b)
```

### **Multiple assignments:**

- Multiple values can be assigned to multiple variables using tuple assignment.

```
>>>(a,b,c)=(1,2,3)
>>>print(a)
1
>>>print(b)
2
>>>print(c)
3
```

### **Tuple as return value:**

- A Tuple is a comma separated sequence of items.
- It is created with or without ( ).
- A function can return one value. if you want to return more than one value from a function. we can use tuple as return value.

#### **Example1:**

```
def div(a,b):
 r=a%b
 q=a//b
 return(r,q)
a=eval(input("enter a value:"))
b=eval(input("enter b value:"))
r,q=div(a,b)
print("remainder:",r)
print("quotient:",q)
```

#### **Output:**

```
enter a value:4
enter b value:3
remainder: 1
quotient: 1
```

#### **Example2:**

```
def min_max(a):
 small=min(a)
 big=max(a)
 return(small,big)
a=[1,2,3,4,6]
small,big=min_max(a)
print("smallest:",small)
print("biggest:",big)
```

#### **Output:**

smallest: 1  
biggest: 6

**Tuple as argument:**

- The parameter name that begins with \* gathers argument into a tuple.

**Example:**

```
def printall(*args):
 print(args)
printall(2,3,'a')
```

**Output:**

```
(2, 3, 'a')
```